UNIVERSIDAD DE MÁLAGA



UNIVERSIDAD
DE MÁLAGA

Sistemas en Tiempo Real

Cipriano Galindo Andrades

Departamento de Ingeniería de Sistemas y Automática

DEPARTAMENTO DE INGENIERÍA
DE SISTEMAS Y AUTOMÁTICA

Origen del Lenguaje C

El lenguaje C fue desarrollado por Dennis Ritchie y Ken Thompson en los laboratorios Bell en 1972.

Inspirado en lenguajes anteriores como el B y el BCPL

Fue desarrollado principalmente para implementar y modificar de forma eficiente el **SO UNIX**

Inicialmente utilizado para desarrollar SO's, aunque en la actualidad se utiliza también para sistemas empotrados.

Principal Ventaja

PORTABILIDAD

Tipos de Lenguajes

Lenguaje Máquina: Cadenas de números que especifican instrucciones máquina:

+1300042774
+1400593419
+1200274027

Lenguaje Ensamblador: Instrucciones abreviadas que representan operaciones elementales:

Load Precio
Add IVA
Store TOTAL

Lenguaje de alto nivel: Instrucciones de alto nivel y notación matemática (necesidad de compiladores)

TOTAL = Precio + IVA

Departamento de Ingeniería de Sistemas y Automática

Lenguaje C

C es un lenguaje estructurado, que permite operaciones de bajo nivel (flexibilidad) y portable.

¿Por qué estudiar C?

- Gran parte de los sistemas de Tiempo Real usan C
- Muchas herramientas software han sido escritas en C
- Ha tenido una gran influencia en otros lenguajes como C#, java, C++, Perl, etc.
- Obliga al programador a entender aspectos fundamentales de la programación de sistemas.

Lenguaje ANSI-C

Desde su aparición surgieron diferentes variantes que en la mayoría de los casos eran incompatibles.

American **N**ational **S**tandards **I**nstitute comenzó a establecer una especificación standard de C. En 1989 se completa el standard que pasa a llamarse C89 o simplemente ANSI-C

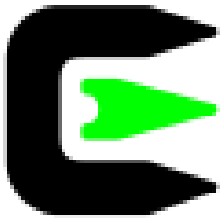
En 1990 el standard ANSI es adoptado por ISO, pasando a conocerse como C90, aunque tanto C89 y C90 son equivalentes.

Última actualización C++20 incluyendo nuevas funcionalidades y actualizaciones.

Lenguaje C

Compilando Lenguaje C







Linux



Editores de Código (algunos integrados en el propio IDE)





Notepadqq

v0.38.0

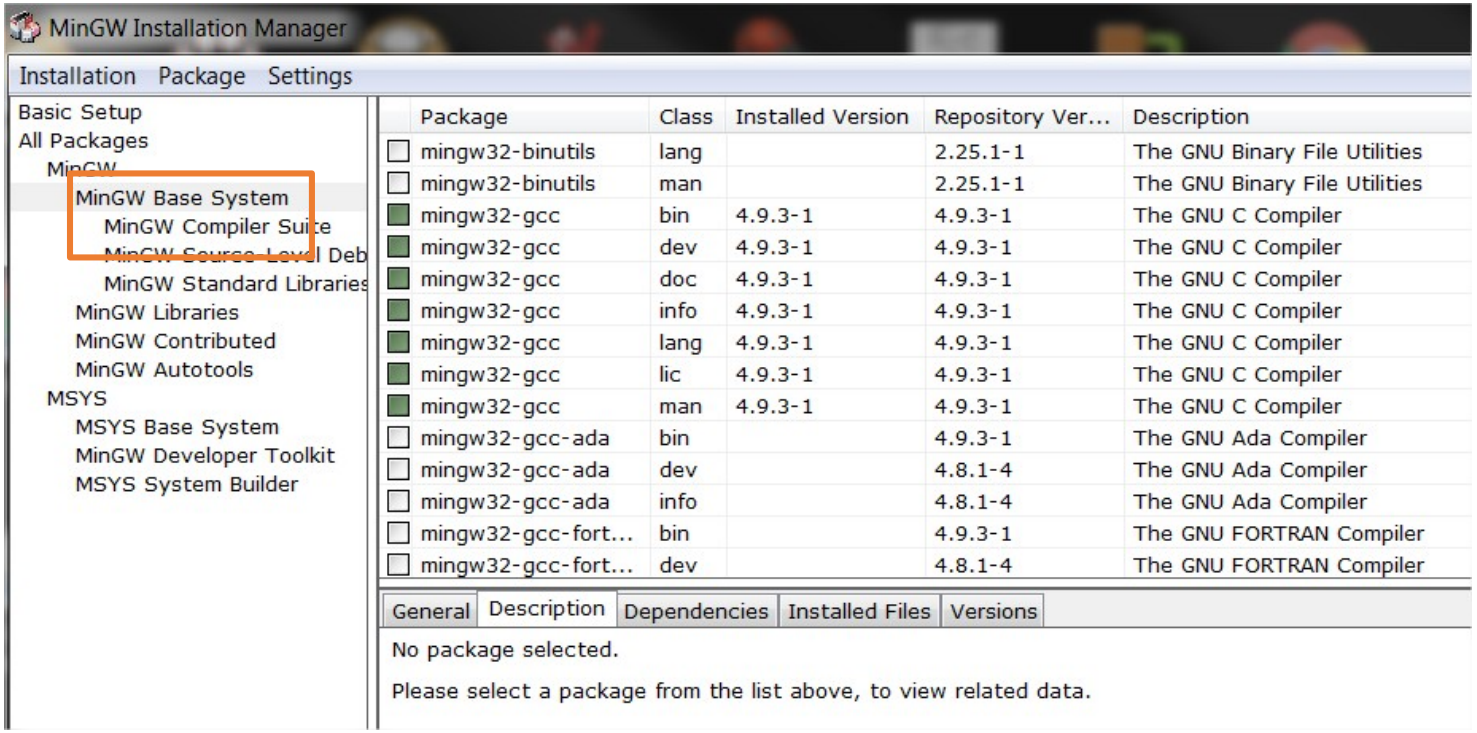
http://notepadqq.altervista.org

Contributors: GitHub Contributors

Copyright © 2010-2014, Daniele Di Sarli

Lenguaje C

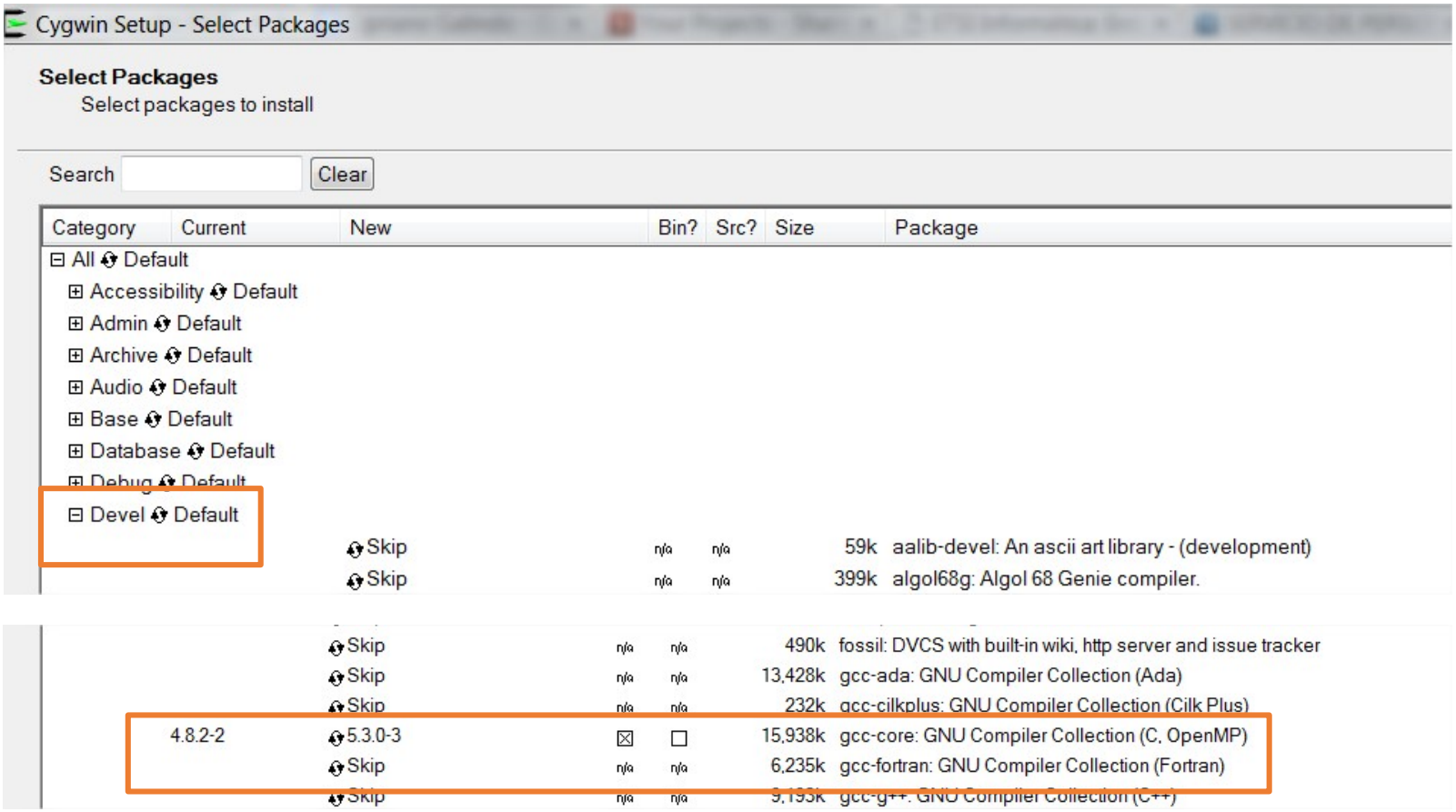
Instalación: Seguir todos los parámetros por defecto y luego seleccionar los mingw32-gcc:



Lenguaje C



Instalación: Seguir todos los parámetros por defecto y seleccionar el paquete **“gcc-core C compiler”** dentro de la sección **“devel”**

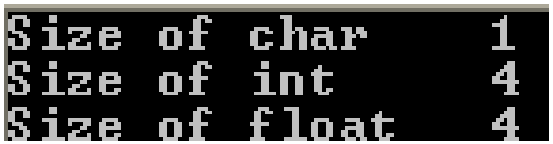


Tipos de Datos Básicos

`char`	Un byte capaz de almacenar un carácter según el conjunto de caracteres local
`int`	Un número entero
`float`	Un número flotante (real) con precisión simple
`double`	Un número flotante (real) con precisión doble

Tipos de Datos Básicos

Diferente tamaño en memoria y por tanto tienen diferente rango



A estos tipos básicos se le añaden varios modificadores:

Modificador de signo [signed]/unsigned

P. ej. `char` representa un entero en el rango [-128,127]

`unsigned char` representa un entero en el rango [0,255]

Modificador de rango short/long/double

Se usan para extender el rango de enteros y flotantes. Típicamente el rango de

```
short <= int <= long
```

```
float <= double <= long double
```

Tipos de Datos Básicos

```
Size of char    1
Size of int     4
Size of float   4
Size of double  8
--------------------------------
Size of short   2
Size of long    4
--------------------------------
Size of unsigned char   1
Size of unsigned int    4
--------------------------------
Max char                127
Max int                 2147483647
Max unsigned char       255
Max unsigned int        4294967295
Max long                2147483647
Max short               32767
Press any key to continue_
```

→Operador sizeof(<tipo>)
→definiciones de máximos/mínimos en limits.h

Variables

Otros modificadores interesantes:

Modificador `const`

P. ej. `const int` representa un entero que no puede ser modificado en el código (una constante)

Modificador `volatile`

Se usa para indicar al compilador que una posición de memoria puede ser cambiada en cualquier momento por un programa externo.

Definición `sig_atomic_t`

Define una variable entera que se accede/modifica de forma atómica incluso en presencia de interrupciones asíncronas. Ejemplo más típico:

```
volatile sig_atomic_t mivariable
```

Variables

Variable.- Un símbolo del código que almacena valores

Identificador.- Secuencia de caracteres que pueden ser letras, dígitos y _ sin que pueda comenzar por un dígito, con un máximo de 31 caracteres, y es "*case sensitive*"

Definición.- Siempre antes de usarlas y siempre antes de la primera instrucción ejecutable.

```
var_type var_name;

int age;
float annual_salary;
double weight, height;
```

Variables

(Asignación de valores)

→En la definición de la variable:

```
int age=25;
char ok='y';
float eps=1.0e-5;
const float gravity=9.8;
```

→En cualquier parte del código mediante instrucciones del tipo:

```
age=25;
annual_salary=age * 100;
```



Por norma general las variables al ser definidas tienen un valor "basura" y por tanto deben ser adecuadamente inicializadas

Constantes

Directivas del compilador (#define)

Una directiva del compilador es una instrucción que es interpretada antes de la generación de código máquina. Sirven típicamente para simplificar la codificación y mejorar su legibilidad.

Ej. directiva `#define`

```
#define numero_de_empleados 45
```

Importante.- No acaba en ; puesto que no es una instrucción

Define una constante no tipada. Solución:

```
#define numero_de_empleados 45L
```

Constantes

Directivas del compilador (#define)

Directiva #define también sirve para definir macros. Pueden llevar parámetros.

```
#define AREA_CIRC(x) PI * (x) *(x)
```

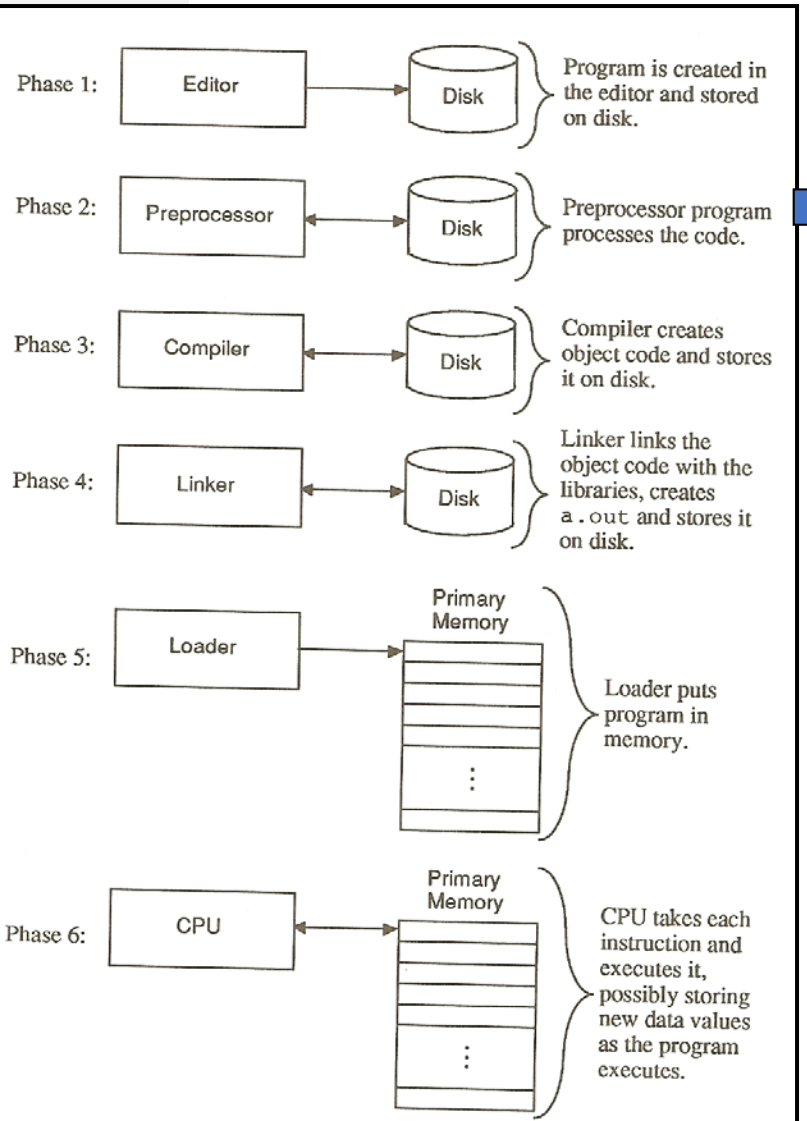
```
a=AREA_CIRC(3);
```
→
```
a=3.14159 * (3) *(3);
```

```
int c=23;
a=AREA_CIRC(c+2);
```
→
```
a=3.14159 * (c+2) *(c+2);
```

```
#define AREA_RECT(a,b) (a)*(b)
```

Departamento de Ingeniería de Sistemas y Automática

Constantes




```
#define numero_de_empleados 45

void main()
{       int n;
        for (n=0;n<numero_de_empleados;n++)
                printf("Empleado #%d\n",n);

}
```

```
void main()
{       int n;
        for (n=0;n<45;n++)
                printf("Empleado #%d\n",n);

}
```

Constantes

Tipos Enumerados. Es una lista de enteros constantes que automaticamente comienzan en cero.

```
enum calificacion {SUSPENSO, APROBADO, NOTABLE, SOBRESALIENTE,
MH};
```

Equivalente a:

```
#define SUSPENSO 0
#define APROBADO 1
#define NOTABLE 2
#define SOBRESALIENTE 3
#define MH 4
```

Constantes

Tipos Enumerados. Se pueden inicializar automáticamente con otro valor:

```
enum calificacion {SUSPENSO=10, APROBADO, NOTABLE, SOBRESALIENTE,
MH};
```

Equivalente a:

```
#define SUSPENSO 10
#define APROBADO 11
#define NOTABLE 12
#define SOBRESALIENTE 13
#define MH 14
```

También se pueden inicializar manualmente:

```
enum calificacion {SUSPENSO='E', APROBADO='D', NOTABLE='C',
SOBRESALIENTE='B', MH='A'};
```

Tipos de Datos Compuestos (Arrays)

Array. Grupo de posiciones contiguas de memoria con el mismo nombre y que hacen referencia a un mismo tipo

```
int lista[5];
```

12	34	55	-3	4

Hay que indicar las dimensiones en la inicialización. Excepción:

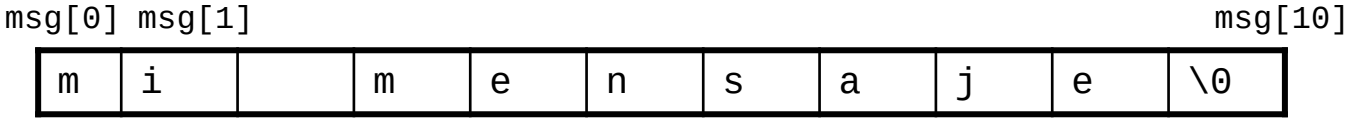
```
int lista[]={12, 34, 55, -3, 4};
char msg[]="Hola mundo";
```

Tipos de Datos Compuestos

Cadenas de Texto.- Son arrays terminados en '\0'

```
char msg[]="mi mensaje"
```

En memoria se almacena como:




```
printf("%c\n",msg[0]);
```

```
m
```

```
printf("%c\n",msg[11]);
```

```
¥
```

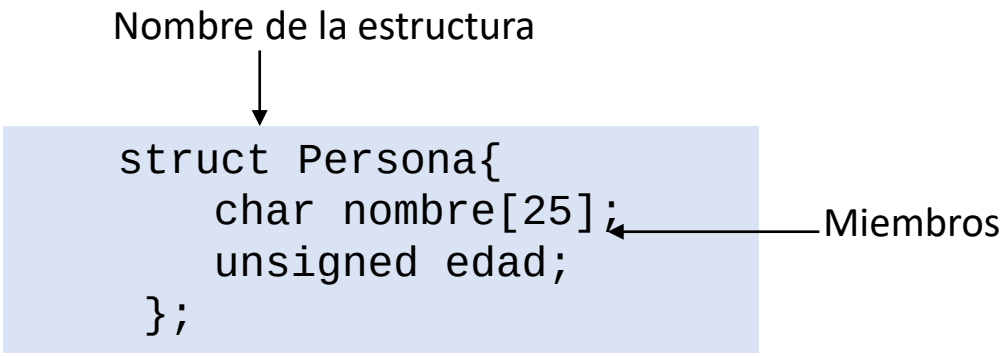


-El compilador no chequea fuera de rango!!! → Cuelgues y comportamiento indeseado

-Diferencia entre `char ok='y'` y `char ok="y"`

Tipos de Datos Compuestos
(Estructuras)

Estructuras.- Colección de variables de diferentes tipos bajo un mismo nombre



Definición de variables de tipo estructura

```
struct Persona p,empleados[20];
```

```
struct Persona{
        char nombre[25];
        unsigned edad;
 } p, empleados[20];
```

Departamento de Ingeniería de Sistemas y Automática

Tipos de Datos Compuestos
(Estructuras)

No se almacenan necesariamente de forma adyacente en memoria!!

```
struct Ejemplo{
    char letra;
    short numero;
 };
```

1byte

2bytes



word 1	00001100	
word 2	00110010	00010001

Va a depender del compilador y de la plataforma HW, aunque todo, será transparente al programador

Tipos de Datos Compuestos (Estructuras)

Inicialización y Acceso a Estructuras

```
struct Persona{
    char nombre[25];
    unsigned edad;
};
```

```
struct Persona p = {“Miguel”, 25};
```

Inicialización en la definición

```
printf(“Nombre: %s\n Edad %d\n”, p.nombre, p.edad);
```

Acceso

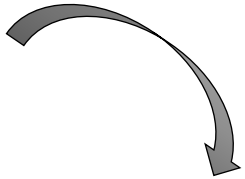
```
p.edad = p.edad + 1;
```

Acceso y modificación

Tipos de Datos Compuestos (Estructuras)

Anidamiento de Estructuras

```
struct Persona{
        char nombre[25];
        unsigned edad;
 };
```



```
struct Empleado{
        struct Persona p;
        char categoria[25];
        unsigned antiguedad;
 };
```

```
struct Empleado e = {{“Miguel”, 25},”becario”,10};
```

```
printf(“Empleado: %s, edad %d, categoria %s, antiguedad %d\n”,
e.p.nombre, e.p.edad, e.categoria, e.antiguedad);
```

Tipos de Datos Compuestos (Uniones)

Uniones. Tipo derivado cuyos miembros ocupan el mismo espacio en memoria, pero sólo un miembro puede ser referenciado cada vez.

Permiten ahorrar espacio en memoria y el compilador reserva espacio para el miembro que ocupe mas.

Nombre de la union

```
union Numero{
    int entero;
    double real;
 };
```

Miembros

```
union Numero n = {3};
```

Inicialización en la definición, sólo para el primer miembro

```
union Numero n = {2.343};
```

Inicialización **INCORRECTA**

Sentencias de Asignación

Asignación de tipos básicos

```
int entero1,entero2;
float real;
char msg[25]= “Hola mundo”;
char msg2[25];

entero1 = 3;
entero2 = 5;
real = 2.3;

entero1 = entero2;
entero1 = real;

msg2 = msg;
```

Definición de variables y posible asignación de valores.

Asignación de valores fuera de la definición

Asignación entre tipos básicos



CUIDADO con los tipos de las variables y con la asignación de cadenas de texto!!

```
asignacion.c:26:7: error: assignment to expression with array type
  msg2 = msg;
       ^
```

Sentencias de Asignación

Asignación de tipos compuestos

NO se pueden asignar tipos compuestos

```
struct Persona
p1={“Miguel”,25};
struct Persona p2;

p2 = p1;
```



Asignación de cada miembro individualmente

```
p2.edad = p1.edad;
strcpy (p2.nombre,p1.nombre);
```

Función para copiar cadenas de texto

Sentencias de Asignación

Operadores de Incremento/Decremento

++ y −− Sirven para incrementar/decrementar variables

```
p1.edad ++;
++ p1.antiguedad;
```

Se pueden usar como operadores prefix o postfix

Aunque tienen un pequeño matiz:

```
n=10;
x=++n;
y=n++;
```

¿Cuánto vale x?

¿Cuánto vale y?

x=11 ; n=11

y=11; n=12

++n incrementa el valor de n antes de ser usado

n++ incrementa el valor de n después de ser usado

Expresiones

Expresión.- Una instrucción del lenguaje que combina variables/constantes con operadores matemáticos/lógicos.

Expresiones Numéricas

```
i = i + 3;
```

```
i += 3;
```

Operadores aritméticos básicos

+	"mas"	→	c = a + b
-	"menos"	→	c = a - b
*	"por"	→	c = a * b
/	"dividido por"	→	c = a/b
%	"modulo de"	→	c = a % b

Precedencia: Paréntesis, * / % , + - , izquierda a derecha

Expresiones

Expresiones Lógicas

Operadores relacionales:

> “mayor que”

< “menor que”

>= “mayor o igual que”

<= “menor o igual que”

Se usan en expresiones condicionales. P.ej. “`if (a<1) then....`”

Pero también se pueden usar para asignar valores: 1 (verdad) o 0 (falso). En general cualquier cosa != 0 se considera verdad

int c, a=2, b=1;

c = (a > b)

¿Cuanto vale c?

Expresiones

Expresiones Lógicas

Operadores de igualdad:

== “igual que”

!= “diferente que”

Operadores de conjunción lógicos:

|| “O lógico”

&& “Y lógico”

```
int c, a=2, b=1;
c = (a== b);
c=(a>b) || (a==b)
c=(a>b) && (a==b)
```



c=0
c!=0
c=0

Expresiones

Operadores de Bits

```
short c;

c = 0x342; //  c=834;
```

& Operación “And”

| Operación “Or”

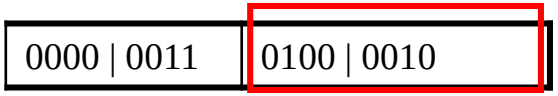
>> Desplazamiento Dcha.

<< Desplazamiento Izq.

^ XOR

~ Complemento a 1

Obtener el byte menos significativo:



b=01000010

b=0x42

```
short c,b;
c = 0x342;
b = c & 0xFF;

c = 834;
b = c & 255;
```

Expresiones

Operadores de Bits

```
short c;

c = 0x342; //  c=834;
```

& Operación “And”

| Operación “Or”

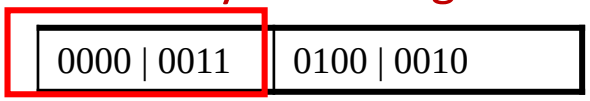
>> Desplazamiento Dcha.

<< Desplazamiento Izq.

^ XOR

~ Complemento a 1

Obtener el byte mas significativo:



b=00000011

b=0x03

```
short c,b;

c = 834;
b = c & 0xFF00;
b = b >> 8;
```

Realmente no haría falta la máscar ya que el desplazamiento añade 0’s

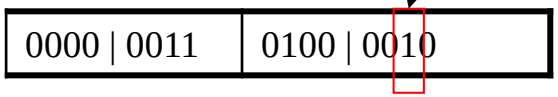
Expresiones

Operadores de Bits

```
short c;

c = 0x342; //  c=834;
```

Poner un bit a 0



b=0x340

& Operación “And”

| Operación “Or”

>> Desplazamiento Dcha.

<< Desplazamiento Izq.

^ XOR

~ Complemento a 1

```
short c,b;

c = 0x342;      {D=1101}
b = c & 0xFFFD;

//de otra forma:
b = c & ~(1<<1);
```

Posición del bit que se quiere poner a 0

Expresiones

Operadores de Bits

```
short c;

c = 0x342; //  c=834;
```

& Operación “And”

| Operación “Or”

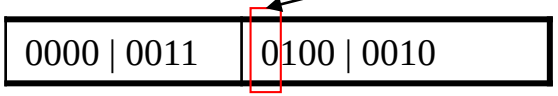
>> Desplazamiento Dcha.

<< Desplazamiento Izq.

^ XOR

~ Complemento a 1

Poner un bit a 1



b=0000001111000010

b=0x3C2

```
short c,b;

c = 0x342;
b = c | 0x80;

b = (1<<7) | c;
```

Posición del bit que se quiere poner a 1

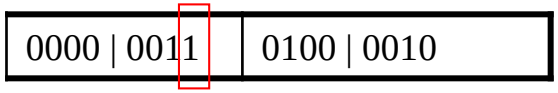
Expresiones

Operadores de Bits

```
short c;

c = 0x342; //  c=834;
```

Averiguar el valor de un bit concreto



b=1

& Operación “And”

| Operación “Or”

>> Desplazamiento Dcha.

<< Desplazamiento Izq.

^ XOR

~ Complemento a 1

```
short c,b;

c = 0x342;
b = (c & (1<<8)) >> 8;
```

Expresiones

Operadores de Bits

```
short c;

c = 0x342; //  c=834;
```

& Operación “And”

| Operación “Or”

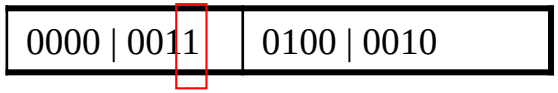
>> Desplazamiento Dcha.

<< Desplazamiento Izq.

^ XOR

~ Complemento a 1

Cambiar el valor de un bit concreto



b=1 → b=0

```
short c,b;

c = 0x342;
b = c ^ (1<<8);
```

Tabla de verdad XOR

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

www.unicrom.com

Sentencias de Flujo de Control (if/else/else if)

```
int a,b,c;
if (a>b)
{
        ...
}
else if (a<b)
{
        ...
}
else
        printf("iguales\n");
```

Condición entre paréntesis

Posibilidad de prescindir de las llaves si prosigue una **única** instrucción

Departamento de Ingeniería de Sistemas y Automática

Sentencias de Flujo de Control (switch)

```
int c,n;
switch (c)
{
  case 0:case 1:
        n++;
        break;
  case 2:
        n+=2;
        break;
  case 3:
        n+=3;
        break;
  default:
        n=0;

}
```

Sólo para expresiones enteras

Etiquetas múltiples

Siempre acabar con break;

Etiqueta default

Sentencias de Flujo de Control (for)

```
int c;
for (c=1;c<10;c++)
{
        printf("%d\n",c);
}
```

Las llaves {} en este ejemplo podrían omitirse

Se repite el bloque entre llaves tantas veces como indique la definición.

3 partes:

c=1 → Asignación inicial

c<10 → Condición de terminación

c++ → Modificación de la vble de control

La condición de terminación se chequea cada vez que comienza una nueva iteración.

Departamento de Ingeniería de Sistemas y Automática

Sentencias de Flujo de Control (for)

```
int c;
int l={2,4,8,3,2}
for (c=0;c<5;c++)
{
        if (l[c]%2)
break;
        ...
}
```

Uso de **break** para salir del bucle for

Si hay varios for anidados, break hace terminar el más interno, volviendo al externo

En el ejemplo, cuando c==3, se ejecuta el break

```
int c;
int l={2,4,8,3,2}
for (c=0;c<5;c++)
{
        if (l[c]%2)
continue;
        printf("PAR\n");
}
```

Uso de **continue** para saltar una iteración

En el ejemplo, cuando c==3, no se imprime "PAR"

Sentencias de Flujo de Control (for)

```
unsigned char c;

for (c=10;c>=0;c--)
{
        printf("%d\n",c);
}
```

¿Qué hace este bucle?

Sentencias de Flujo de Control (while)

```
int c;
c=1;
while (c<10)
{
        printf("%d\n",c);
        c++;
}
```

Se repite el bucle mientras se cumpla la condición

Error típico: Olvidar la modificación de la variable de control

Equivalencia while/for

```
int c;
for (c=1;c<10;c++)
{
        printf("%d\n",c);
}
```

Bucles sin fin

```
while (1);
```

```
for (;;);
```

```
do
while(1);
```

Departamento de Ingeniería de Sistemas y Automática

Sentencias de Flujo de Control (do..while)

```
int c;
c=1;
do
{
        printf("%d\n",c);
        c++;
}while (c<10)
```

Similar a while.- El bucle se ejecuta mientras se cumpla la condición

Diferencia.- La condición se chequea al final de cada iteración (no al principio), con lo que al menos se ejecutan una vez

Funciones

Permiten la división del código en bloques, facilitando su legibilidad, mantenimiento y trazabilidad.

Se definen:

```
Tipo-de-retorno nombre-de-la-funcion ( parametros)
{
        //Cuerpo de la función
}
```

Ejemplo de función que no hace nada:

```
dummy(){}
```

Funciones

Función que realiza la suma de dos enteros pasados como parámetros

```
int Suma (int a, int b)
{
        int c;

        c=a+b;

        return c;
}
```

Definición de variables locales

Cuerpo de la función

Devolución de un valor

Funciones

```
int error=2;

int Suma2 (int a, int b);

int main(void)
{
        int a=3,b=5;
        int s;
        s=Suma2(a,b);
        printf("%d\n",s);
        return 0;
}
int Suma2 (int a,int b)
{
        int c;
        c=a+b-error;
        return c;
}
```

Variable Global

Prototipo de la función

Función especial main()

int main (int argc, char* argv[])

void main(void)

Variables Local

Uso de la variable global

Funciones

```
int error=2;

int Suma2 (int a, int b);

int main(void)
{
        int a=3,b=5;
        int s;
        s=Suma2(a,b);
        printf("%d\n",s);
        return 0;
}
int Suma2 (int a,int b)
{
        int c;
        c=a+b-error;
        return c;
}
```

Consideraciones

→Los paréntesis son obligatorios para pasar parámetros

→No se puede hacer "return" de tipos compuestos

→Ineficiencia al pasar tipos compuestos como parámetros (mejor pasarlos por referencia)

→Volatilidad de las variables locales (uso de static)

→Responsabilidad del llamante sobre el qué hacer con el valor retornado

Funciones

```
int error=2;

int Suma3 (int a, int b,int *c);

int main(void)
{
        int a=3,b=5,c;
        int s;
        s=Suma3(a,b,&c);
        printf("%d,%d\n",s,c);
        return 0;
}
int Suma3 (int a,int b, int *c)
{
        int d;
        d=a+b-error;
        *c=a+b+error;
        return d;
}
```

Si necesitamos pasar tipos compuestos a una función, o que ésta devuelva más de un valor, podemos usar paso de parámetros por referencia

Se le pasa directamente la dirección de memoria donde está alojada la variable mediante el **operador &**

Cualquier modificación hecha por la función, queda reflejada en la variable declarada en el main

Módulos

Cuando el código a implementar es muy complejo y/o participan diferentes personas, es conveniente separarlo en módulos.

Cabecera → Ficheros .h	Definición de tipos y de los prototipos de las funciones para ser utilizadas externamente Uso de la directiva: `#include`
Código → Ficheros .c	Implementación de las diferentes funciones que componen el programa

NADA DE CÓDIGO EN LOS .h!

Módulos

```
// Comentario
/* Mis Funciones Matematicas
   Fichero de definicion Mis_Mat.h
*/

#define MI_PI 3.1415

float circ_area(float radio);
float rect_area(float a, float b);
```

```
// Comentario
/* Mis Funciones Matematicas
   Fichero de implementacion Mis_Mat.c
*/

#include “Mis_Mat.h”

float circ_area(float radio)
{
            return PI*radio*radio;
}
float rect_area(float a, float b)
{
            return a*b;
}
```

```
// Mi programa


#include “Mis_Mat.h”
#include <stdio.h>

int main(void)
{
     float radio=3.4;
     float lado=2.1;
     float area1,area2;

        area1=circ_area(radio);
        area2=rect_area(lado,lado);

        printf(“%f,%f\n”,area1,area
2);
        return 0;

}
```

Módulos

Compilación Separada:

```
>>gcc –c Mis_Mat.c –o Mis_Mat_obj.o
```



Mis_Mat_obj.o

```
>>gcc  main.o Mis_mat_obj.o  –o program
```



program

Otra opcion:

```
>>gcc Mis_Mat.c main.c –o program
```

```
>>gcc Mis_Mat.c main.c –o program –std=C90
```

Módulos

```
#include <stdio.h>

printf (“ Texto probablemente con formato “,variables);

sprintf (variable,“Texto probablemente con formato “,variables);

scanf (“Formato “,variables);

sscanf (variable,“Formato “,variables);
```

Sequencias de escape (carácter ‘\’)

\n	Nueva linea	\a	Beep
\t	Tabulador horizontal	\\	\
\r	Retorno de carro	\”	“

Módulos

```
#include <stdio.h>
```

%d	Entero	**%6d**	Entero con 6 dígitos
%f	Flotante	**%6f**	Flotante con 6 dígitos
%l, %h	Long, Short	**%.2f**	Flotante con 2 decimales
%u	Unsigned	**%6.2f**	Flotante con 6 dígitos y 2 decimales
%c	Character	**%s**	String

```
#include <stdlib.h>

long int strtol (const char* str, char** endptr, int base);
```

Convierte a decimal una cadena que representa un número en una determinada base

Módulos

Otros módulos

```
#include <string.h>

        strcpy (cad_destino,cad_origen);
        int strlen (cadena);
        int strcmp (cadena1,cadena2);
```

Tipos de Usuario

Definición de nuevos tipos

Construcción `typedef`

Sirve para crear un sinónimo de un tipo de dato previamente creado.
Se suele usar para crear nombres más cortos para las estructuras.

```
typedef struct Persona persona;
persona p;
```

También se usa con tipos simples

```
typedef int Mi_Integer;
typedef long Mi_Integer;
```

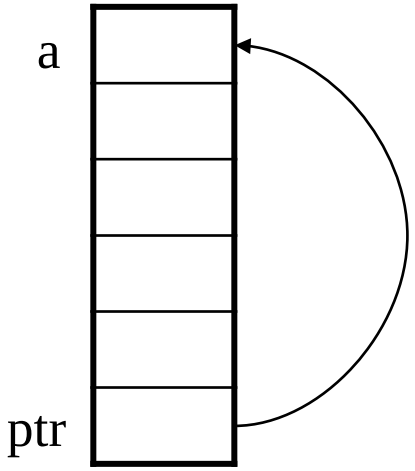
Punteros

Puntero

Son variables que almacenan direcciones de memoria.

Un int, p. ej. almacena un valor. Un puntero almacena la dirección de memoria donde está ese int.

```
int a;
a=5;
```



ptr apunta a una variable de tipo int que vale 5. El valor de ptr es la dirección de **a** en memoria.

Punteros

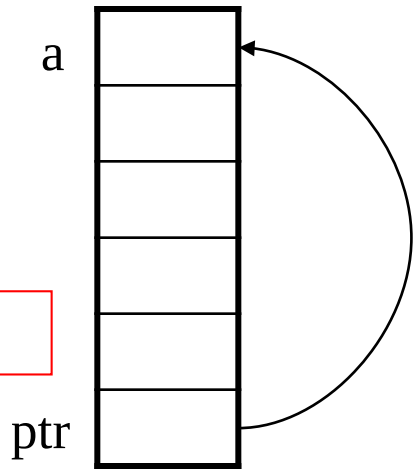
Declaración de Punteros

Los punteros han de ser declarados y son variables tipadas.

Peligro: Usar un puntero sin haberlo inicializado correctamente.

```
int a;
a=5;
int * ptr;
ptr=&a;
```

& significa “dirección de”



Se asigna a ptr la dirección de la variable a.

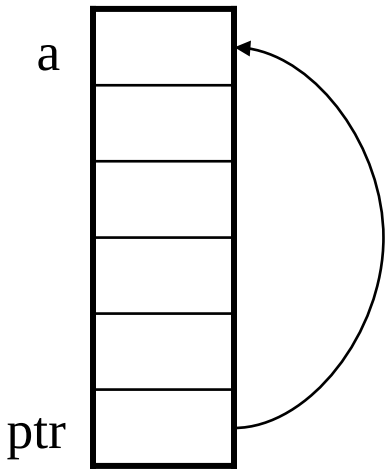
ptr es un puntero a int. El compilador usa esta información para recuperar su valor

Punteros

Acceso a los datos apuntados

Se accede con el símbolo *

```
int a;
a=5;
int * ptr;
ptr=&a;
*ptr=7;
printf("%d\n",a);
printf("%d\n",*ptr);
```



Mediante *ptr se accede a la variable apuntada. Como se ha declarado de tipo int, *ptr puede ser utilizado como un int.

* significa "lo apuntado por"

Punteros

Punteros a estructuras y uniones

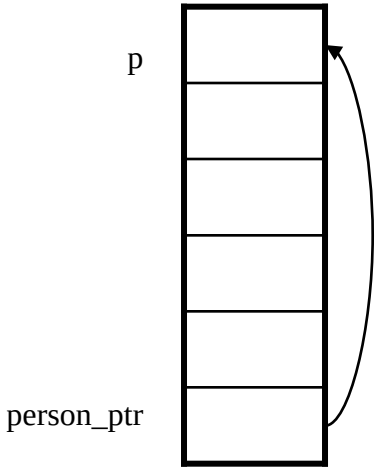
Se accede con el símbolo ->

```
struct Persona{
        char nombre[25];
        unsigned edad;
 };

struct Persona p;
struct Persona *person_ptr;

person_ptr=&p;
(*person_ptr).edad=25;

person_ptr->edad=25;
strcpy(person_ptr->nombre,“Miguel”);
```



Punteros

Inicialización de punteros a zonas de memoria no reservadas.

Existen funciones especiales para reservar/liberar memoria dinámica del heap para punteros.

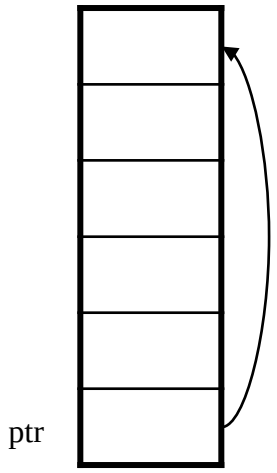
```
#include <alloc.h>

int *ptr;

ptr=malloc(sizeof(int));

*ptr=5;

free(ptr);
```



Reserva espacio para almacenar un entero.

Lo libera, pero no borra el direccionamiento.

Punteros

Punteros que no apuntan a nada

Son puntero nulos (NULL) y se suelen usar para inicializar punteros y para chequear situaciones anómalas.

```
int *ptr;

ptr=NULL;

…

if (ptr==NULL)
        ptr=malloc(sizeof(int));
else
        {
         free(ptr);
         ptr=NULL;
        }
```

Punteros

Punteros que no llevan tipo

Tipo void *, indica un puntero a cualquier cosa. Es obligación del programador saber a qué tipo realmente apunta para poder trabajar con ellos.

```
void * GetNumber(short n)
{
        int *ptri;
        float *ptrf;
        void *res;
        if (n==0)
          { ptri=(int *)malloc(sizeof(int));
          *ptri=5;
          res=(void*)ptri;
           }
        else
          {ptrf=(float *)malloc(sizeof(float));
          *ptrf=5.23;
          res=(void*)ptrf;
           }
        return res;}
```

```
void *res,*res1;
int a;
float b;

res=GetNumber(0);
res1=GetNumber(1);

a=* (int*)res;

b=* (float*)res1;
```

Punteros

Aritmética de Punteros

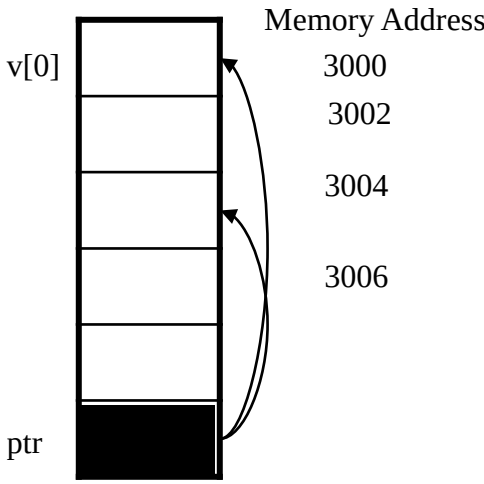
Se puede operar (aritmeticamente) con las direcciones de memoria apuntadas por un puntero. Útil para trabajar con arrays

```
int v[5];

int* ptr;
ptr=&v[0];
```

```
ptr+=1;
```

```
printf("%d\n",*(ptr-1));
```



Equivalencia entre punteros y arrays: El nombre de un array (sin []) es en realidad un puntero al primer elemento del array. **v==&v[0]**

Punteros

Uso de Punteros para el paso de parámetros

-Útiles para el paso y retorno de parámetros complejos

-Uso para paso de parámetros por referencia

```
int GetPersonAge(struct Person p)
{
        return p.edad;
}
```

Ineficiente.-Se copia la estructura en cada llamada

```
int GetPersonAge(const struct Person
*p)
{
        return p->edad;
}
```

Eficiente.-Se pasa sólo la dirección de la estructura en cada llamada (uso de const)

```
int GetPersonAge(struct Person *p)
{
        p->edad++;
        return p->edad;
}
```

Paso por referencia.- Se puede modificar el valor del parámetro

Punteros

Punteros a funciones

Se usan para pasar funciones como parámetro a otras funciones. El nombre de una función es en realidad un puntero a su código

```
int Mi_Compare(int a, int b)
{
    if (a>b) return a;
       else return 0;
}
```

```
int Ordena (int* lista, int
(*funct)(int,int))
{
       if (funct(*lista,*(lista+1)))
       …

}
```

```
main()
{
       int v[]={4,5,1,4};
       Ordena(v,Mi_compare);
}
```

Punteros

Punteros a funciones

También se pueden usar para asignar funciones a una variable

```
int Mi_Compare(int a, int b)
{
    if (a>b) return a;
       else return 0;
}
```

```
int Ordena (int* lista, int
(*funct)(int,int))
{
       if (funct(*lista,*(lista+1)))
       …

}
```

```
main()
{
       int v[]={4,5,1,4};
       int (*f)(int,int);

       f=Mi_compare;
       Ordena(v,f);
}
```